

Diseño, Implementación y Arquitectura Modular de Funciones

Asignatura: Programación I / POO **Nivel:** Intermedio - Avanzado **Modalidad:** Individual / Duplas

Objetivos de Aprendizaje:

Implementar funciones puras con firmas flexibles usando parámetros opcionales y valores por defecto.

Utilizar métodos estáticos (@staticmethod) para desacoplar utilidades de dominio de instancias de clase.

Dominar la empaquetación/desempaquetación de argumentos variables mediante *args y **kwargs.

Orquestrar la interacción inter-clases mediante delegación de mensajes y composición de objetos.

EJERCICIOS PRÁCTICOS

Ejercicio 1: Funciones puras con parámetros opcionales y keyword arguments

CONCEPTO: PARÁMETROS

OPCIONALES

Contexto: En un sistema de e-commerce, el cálculo de facturación requiere aplicar impuestos, descuentos y recargos opcionales sin duplicar código ni forzar al cliente a pasar todos los argumentos siempre.

Consigna: Implementá la función calcular_factura_final siguiendo exactamente la firma y comportamiento especificados:

```
def calcular_factura_final(monto_base: float, impuesto: float = 21.0, descuento: float = 0.0, envio_prioritario: float | None = None) -> float:  
    # Retorna el importe total final procesado
```

Requerimientos Técnicos:

Calcular primero el monto con descuento: $\text{monto_base} * (1 - \text{descuento} / 100)$.

Aplicar el impuesto sobre el monto descontado: $\text{subtotal} * (1 + \text{impuesto} / 100)$.

Si envio_prioritario no es None, sumar dicho valor fijo al total obtenido.

Retornar el valor numérico redondeado a 2 decimales.

PRUEBAS OBLIGATORIAS A EJECUTAR:

1. calcular_factura_final (1000.0) → Esperado: 1210.0

2. calcular_factura_final (1000.0, descuento=10.0) → Esperado: 1089.0

3. calcular_factura_final (1000.0, impuesto=10.0, descuento=5.0, envio_prioritario=150.0)

→ Esperado: 1195.0

Ejercicio 2: Métodos Estáticos (@staticmethod) como Librería de Utilidades

CONCEPTO:

@staticmethod

Contexto: En aplicaciones financieras es conveniente agrupar validaciones y conversiones matemáticas en clases de utilidad sin necesidad de instanciar objetos repetidamente en memoria.

Consigna: Diseña la clase ValidadorFinanciero que funcione exclusivamente como un contenedor estático.

Requerimientos Técnicos:

La clase no debe poseer método `__init__`.

Implementar `@staticmethod es_cuit_valido(cuit: str) -> bool`: Verifica que el string contenga exactamente 11 dígitos numéricos (usar `isdigit()` y validación de longitud).

Implementar `@staticmethod convertir_moneda(monto: float, tasa_cambio: float, comision: float = 0.02) -> float`: Convierte el monto multiplicándolo por la tasa y descontando el porcentaje de comisión indicado.

```
class ValidadorFinanciero: @staticmethod def es_cuit_valido(cuit: str) -> bool:
```

Lógica de validación pass

PRUEBAS OBLIGATORIAS (SIN INSTANCIAR CON `()`):

```
print(ValidadorFinanciero.es_cuit_valido("20384920194")) → True
```

```
print(ValidadorFinanciero.es_cuit_valido("20-38492019-4")) → False
```

```
print(ValidadorFinanciero.convertir_moneda(100.0, 1000.0, comision=0.05)) →  
95000.0
```

Ejercicio 3: Interacción Inter-Clase, Métodos de Instancia y Delegación

CONCEPTO: COLABORACIÓN DE OBJETOS

Contexto: En un sistema transaccional, la clase encargada de procesar pagos delega la emisión de comprobantes a un objeto notificador independiente.

Consigna: Creá las clases Notificador y ProcesadorPagos para simular el cobro de un carrito de compras.

Requerimientos Técnicos:

Clase Notificador: Posee el método `enviar_recibo(self, cliente: str, total: float) -> None` que imprime un resumen formal del cobro.

Clase ProcesadorPagos:

En su `__init__` recibe o instancia un objeto de tipo Notificador.

Método `procesar_transaccion(self, cliente: str, items: list[dict], descuento_cupon: float = 0.0) -> float`.

La función itera los items (cada uno un dict con claves "nombre" y "precio"), suma los precios, aplica el descuento opcional de cupón y llama a `enviar_recibo` del notificador.

PRUEBA DE INTEGRACIÓN:

```
carrito = [{"nombre": "Teclado", "precio": 50.0}, {"nombre": "Mouse", "precio": 30.0}]
```

```
procesador = ProcesadorPagos()
procesador.procesar_transaccion("Ana Gómez", carrito, descuento_cupon=10.0)
```

Ejercicio 4: Manejo de Aridad Variable (*args y **kwargs) CONCEPTO: *ARGS / **KWARGS

Contexto: Los subsistemas de logging deben registrar una cantidad indeterminada de eventos de texto junto a metadatos de contexto variables (IP, usuario, tiempo de ejecución, etc.).

Consigna: Implementa la función generar_auditoria_sistema capaz de recibir cualquier número de mensajes y metadatos clave-valor.

```
def generar_auditoria_sistema(modulo: str, *mensajes: str, **metadatos) -> str:
    # Retorna un reporte formateado multi-línea
```

Requerimientos Técnicos:

modulo: Parámetro posicional obligatorio que se formateará en mayúsculas.

*mensajes: Captura N líneas de log y las numera secuencialmente ([1] ... [2] ...).

**metadatos: Captura N pares clave=valor y los desglosa en el formato CLAVE: VALOR.

PRUEBA REQUERIDA:

```
log = generar_auditoria_sistema("AUTH", "Intento fallido", "Bloqueo de IP",
    usuario="admin", ip="192.168.1.10")
```

Ejercicio 5: Sistema Integrador (POO, Estáticos, Métodos y Kwargs)

CONCEPTO: INTEGRACIÓN TOTAL

Contexto: Integración final de componentes para una plataforma de salud y rendimiento deportivo.

Consigna: Desarrolla las clases CalculadoraFitness y Atleta.

Requerimientos Técnicos:

CalculadoraFitness (Clase de utilidades estáticas):

```
@staticmethod calcular_imc(peso_kg: float, altura_m: float) -> float (Fórmula: peso /
    (altura^2)).
```

```
@staticmethod clasificar_nivel(imc: float) -> str ("Bajo peso" < 18.5, "Normal" < 25.0,
    "Sobrepeso" ≥ 25.0).
```

Atleta (Clase de entidad):

```
__init__(self, nombre: str, peso: float, altura: float)
```

```
obtener_reporte(self, incluir_recomendacion: bool = False, **metricas_extra) -
    > str
```

El método obtener_reporte llama a los métodos estáticos de CalculadoraFitness, adjunta las métricas dinámicas enviadas por **metricas_extra y agrega una recomendación opcional.